

Phân tích thiết kế hướng đối tượng

Bài 6: Phân tích đối tượng

TS. Nguyễn Hiếu Cường

Bộ môn Kỹ thuật phần mềm & Hệ thống số

Khoa CNTT, Trường ĐH GTVT

Phân tích hệ thống

- Mô hình ca sử dụng
 - Hệ thống cần làm gì (dưới góc nhìn của người dùng)
 - Các “chức năng” mà hệ thống cần thực hiện được
 - Nhưng, **phần mềm bao gồm các đối tượng**
 - Dựa vào đâu để xác định các đối tượng?
 - Mô tả vấn đề, mô tả bài toán, đặc tả yêu cầu, ...
 - Phân tích danh từ trong đặc tả ca sử dụng
-

Khái niệm về phân tích đối tượng

- Là quá trình xây dựng mô hình miền bài toán (problem domain) bằng các “thuật ngữ” của đối tượng
 - Là giai đoạn "**Làm mô hình cho bài toán**", chưa phải giai đoạn "Làm mô hình cho giải pháp phần mềm"
 - Các việc cần làm trong phân tích đối tượng?
 - Xác định các khái niệm/ thực thể (entities) từ các ca sử dụng
 - Xây dựng sơ đồ lớp phân tích (class diagram) → Xác định “cái gì” có trong bài toán
-

Đầu vào/đầu ra của phân tích đối tượng

- Đầu vào
 - Các kịch bản ca sử dụng, đặc tả yêu cầu
 - Các tài liệu hiện có (biểu mẫu, báo cáo, quy trình hiện hành)
 - Đầu ra
 - Danh sách ban đầu các đối tượng tiềm năng (candidate objects), sau khi sàng lọc (loại bỏ danh từ trừu tượng, trùng lặp, không liên quan)
 - Sơ đồ lớp phân tích lớp (khái niệm) – (conceptual) class diagram
-

Mục tiêu của phân tích đối tượng

- Trong bước này cần:

1. Xác định các lớp khái niệm trong miền bài toán
2. Ghi nhận các thuộc tính (attributes) quan trọng của các lớp
3. Xác định quan hệ giữa các khái niệm giữa các lớp

- Nhưng chưa cần:

1. UI, database, hay coding chi tiết
 2. Đi quá sớm vào các chi tiết kỹ thuật hoặc thiết kế trong giai đoạn phân tích (Thiết kế sớm) → Dễ bỏ sót lớp quan trọng
-

Ví dụ

- Một cô gái (Suzy) có:
 - Nuôi các con vật làm cảnh trong một chuồng (gồm nhiều ngăn)
 - Mỗi ngăn chỉ nuôi đúng 1 con vật (chó hoặc mèo), mỗi con có tên gọi
 - Số lượng ngày càng nhiều và thường xuyên thay đổi (thêm, bớt)
 - Suzy cần một chương trình để quản lý các con vật trên
 - Chương trình có các nhiệm vụ gì? (use cases)
-

Phân tích

- Các đối tượng gì trong bài toán

- Các con vật (chó hoặc mèo)
- Cái chuồng

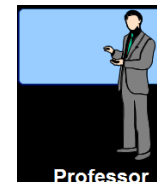
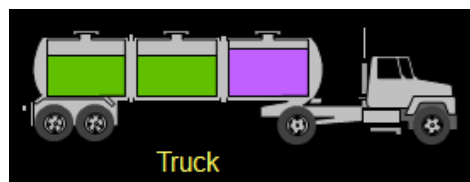
→ Các lớp cần có:

- Animal Mô tả các đặc tính chung của các loại con vật
 - Cat Mô tả loại đối tượng con mèo, kế thừa lớp Animal
 - Dog Mô tả loại đối tượng con chó, kế thừa lớp Animal
 - Kennel Chuồng nuôi các con vật, gồm nhiều ngăn
-

Đối tượng

- Thế giới thực bao gồm các *đối tượng* (object)!

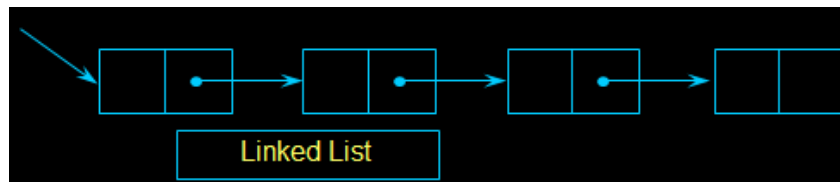
- Đối tượng vật lý



- Đối tượng khái niệm



- Đối tượng phần mềm



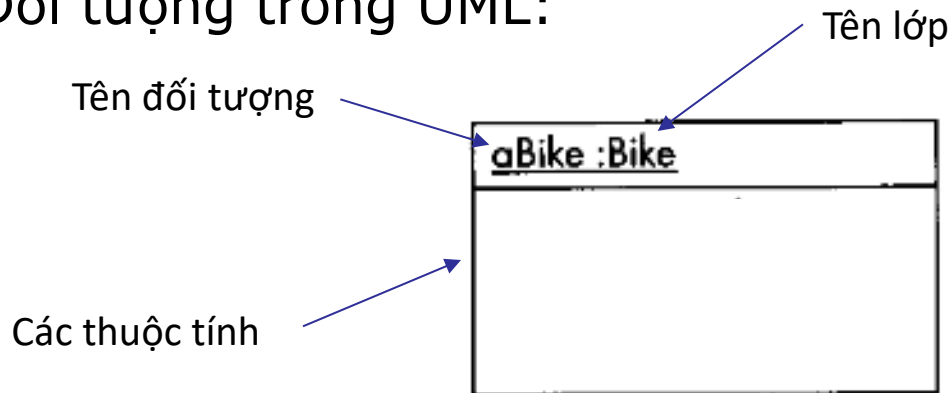
- Mỗi đối tượng gồm các *thuộc tính* và các *thao tác*

Lớp

- Lớp (class) là định nghĩa trừu tượng của các đối tượng có cùng những đặc tính chung
 - Lớp có tác dụng
 - Trừu tượng hoá dữ liệu (data abstraction)
 - Bao gói thông tin (encapsulation)
 - Che giấu thông tin (information hiding)
 - Lớp là khuôn mẫu ("template") cho đối tượng
 - Đối tượng là thể hiện cụ thể (instance) của một lớp
-

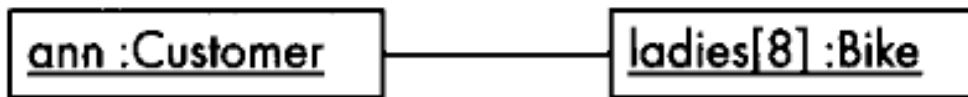
Đối tượng

- Một đối tượng gồm những gì?
 - Trạng thái (state): định hình bởi giá trị các thuộc tính của đối tượng
 - Ứng xử (behaviour): thể hiện bởi các hành động có thể của đối tượng
 - Định danh (identity): mỗi đối tượng là duy nhất trong bộ nhớ
- Đối tượng trong UML:

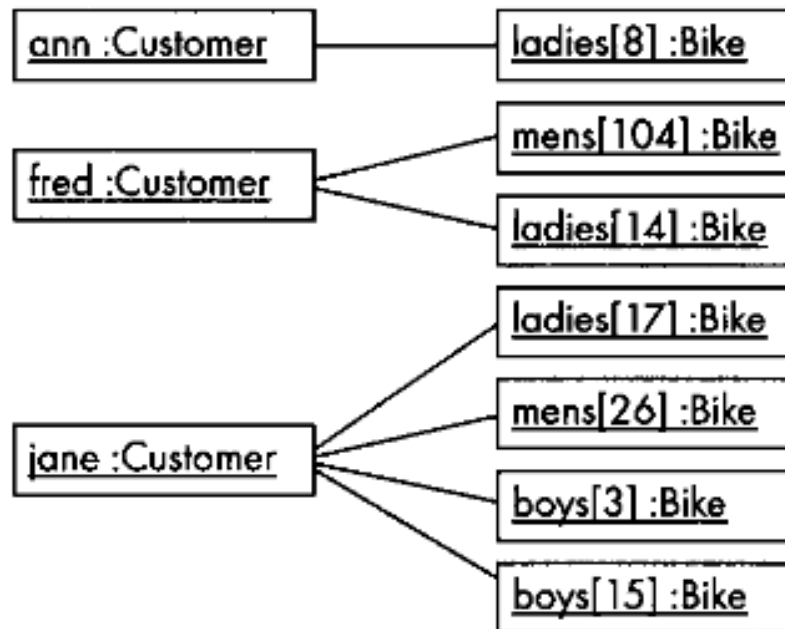


Sự phụ thuộc giữa các đối tượng

- Đối tượng A phụ thuộc vào đối tượng B: A sử dụng các dịch vụ cung cấp bởi B
- Ví dụ: Khách hàng tên ann (đối tượng của lớp Customer) thuê xe đạp ladies[8] (đối tượng của lớp Bike)

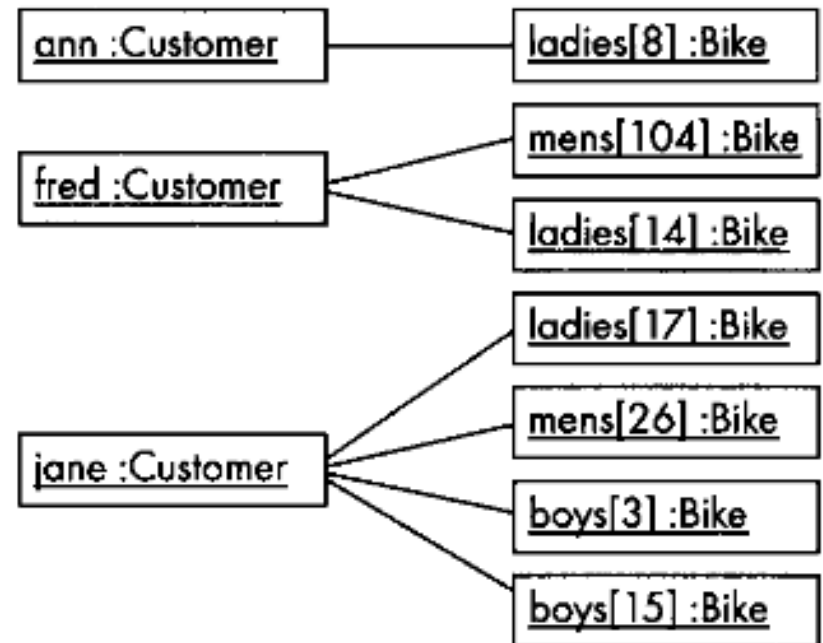


Sơ đồ đối tượng (Object diagram)



Ưu nhược điểm

- Ưu điểm
 - Mô tả quan hệ giữa các đối tượng
 - Làm rõ hơn các hệ thống phức tạp **tại một thời điểm**
- Nhược điểm?

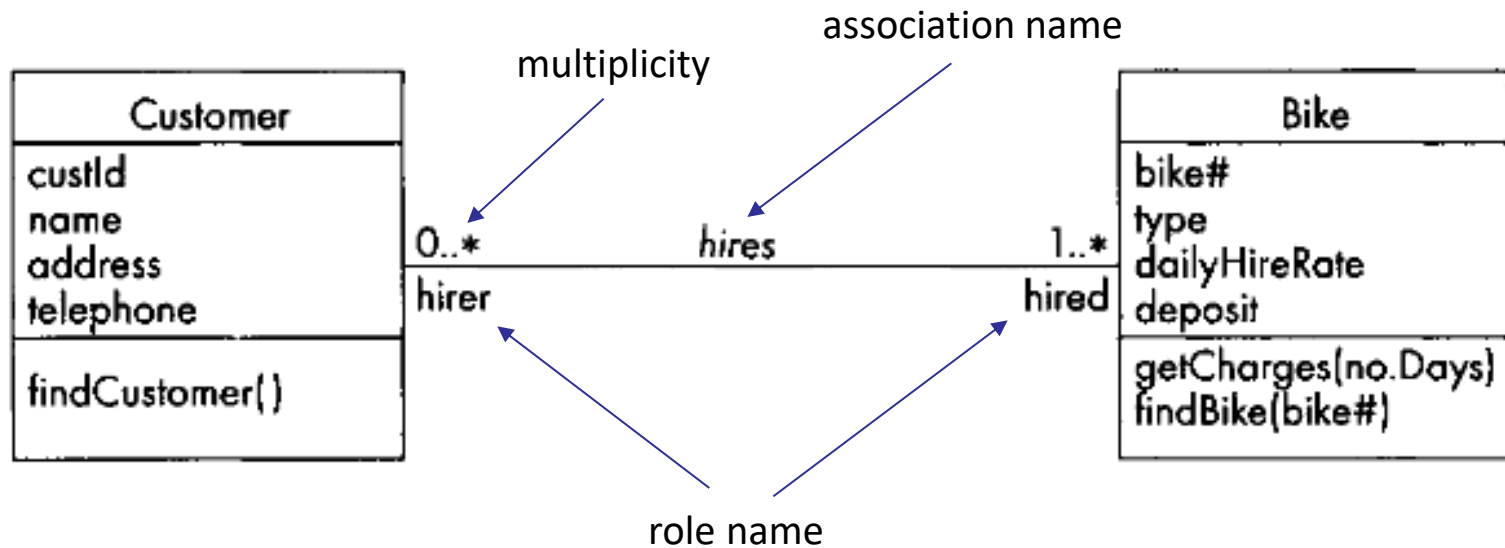


Sơ đồ lớp (Class diagram)



Liên kết giữa các lớp

- Lớp A liên kết với lớp B → Các đối tượng của A có thể tương tác được với các đối tượng của B



Các kiểu liên kết

Có nhiều kiểu liên kết giữa các lớp:

1. Kết hợp (association)

Liên kết căn bản và phổ biến nhất (dạy, học, mượn, trả, ...)

2. Kết tập (aggregation)

Liên kết "tổng thể - thành phần"

3. Gộp (composition)

Liên kết "tổng thể - thành phần", nhưng "chặt" hơn

4. Kế thừa (inheritance)

Liên kết "là một" hoặc "là một loại"

Kết hợp (Association)

- Các đối tượng của hai lớp có thể tương tác với nhau (thông qua truyền thông điệp – message passing)
- Tên liên kết có thể được ghi rõ (nếu cần)



Liên kết bội (Multiplicity)

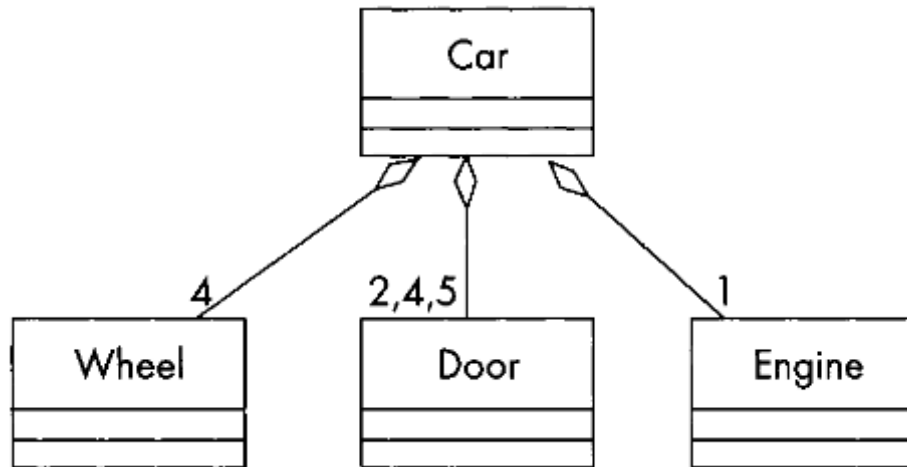
- Xác định giới hạn số đối tượng có thể tham gia vào liên kết



<i>Meaning</i>	<i>Example</i>	<i>Notation</i>
an exact number	exactly one exactly six	1 (or may be omitted) 6
many	zero or more one or more, lots of	0..* 1..*, *
a specific range	one to four, zero to six	1..4, 0..6,
a choice	two or four or five	2, 4, 5

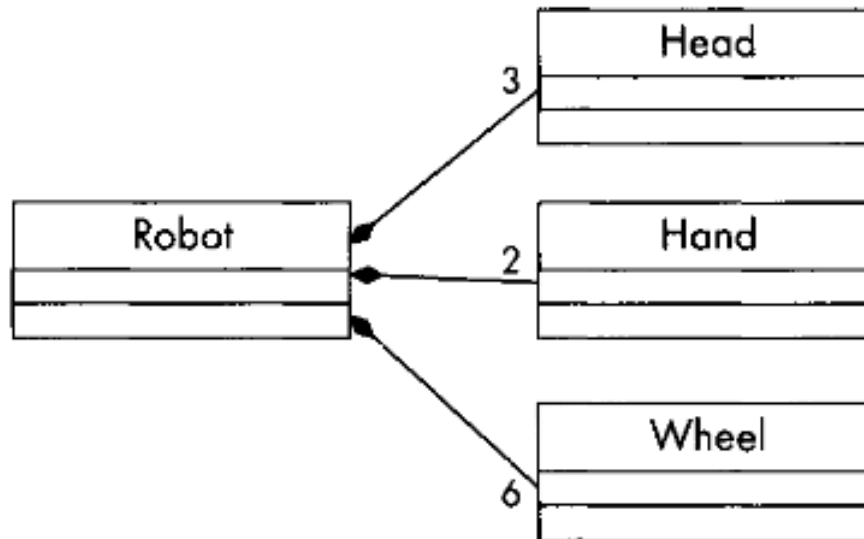
Kết tập (Aggregation)

- Thể hiện quan hệ “tổng thể - thành phần” (whole-part)
 - Trong mô tả thường có các cụm từ: “là một phần của” (is a part of) hoặc “gồm có” (consist of)

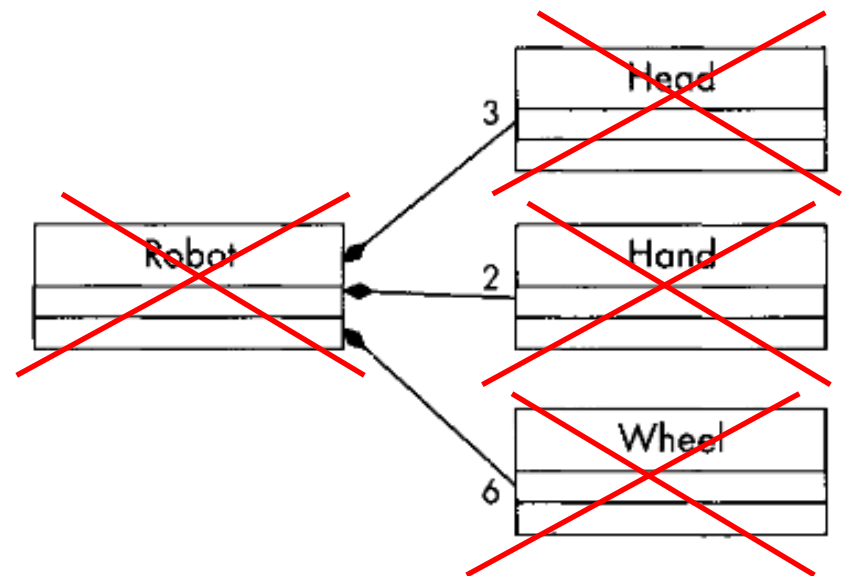
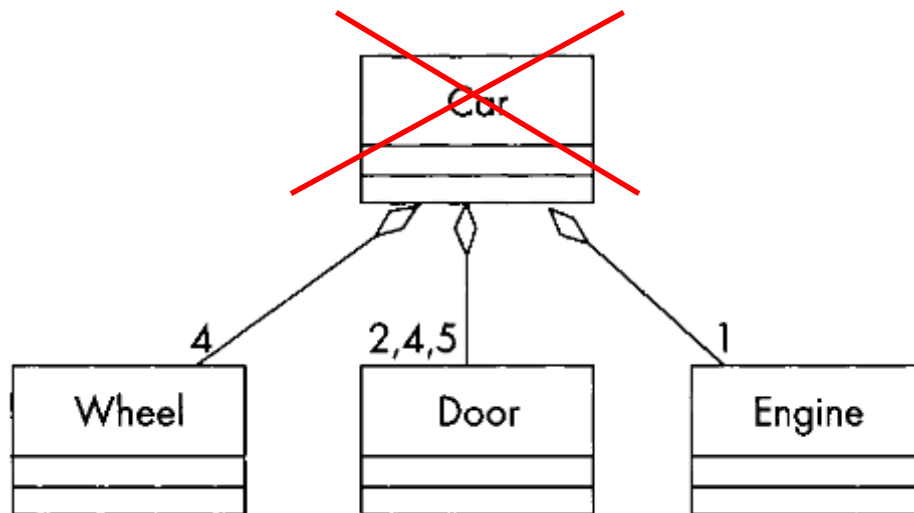


Gộp (Composition)

- Là một dạng quan hệ “tổng thể - thành phần”
 - “Chặt” hơn quan hệ kết tập: đối tượng thành phần không thể tồn tại độc lập với đối tượng tổng thể



Sự khác nhau

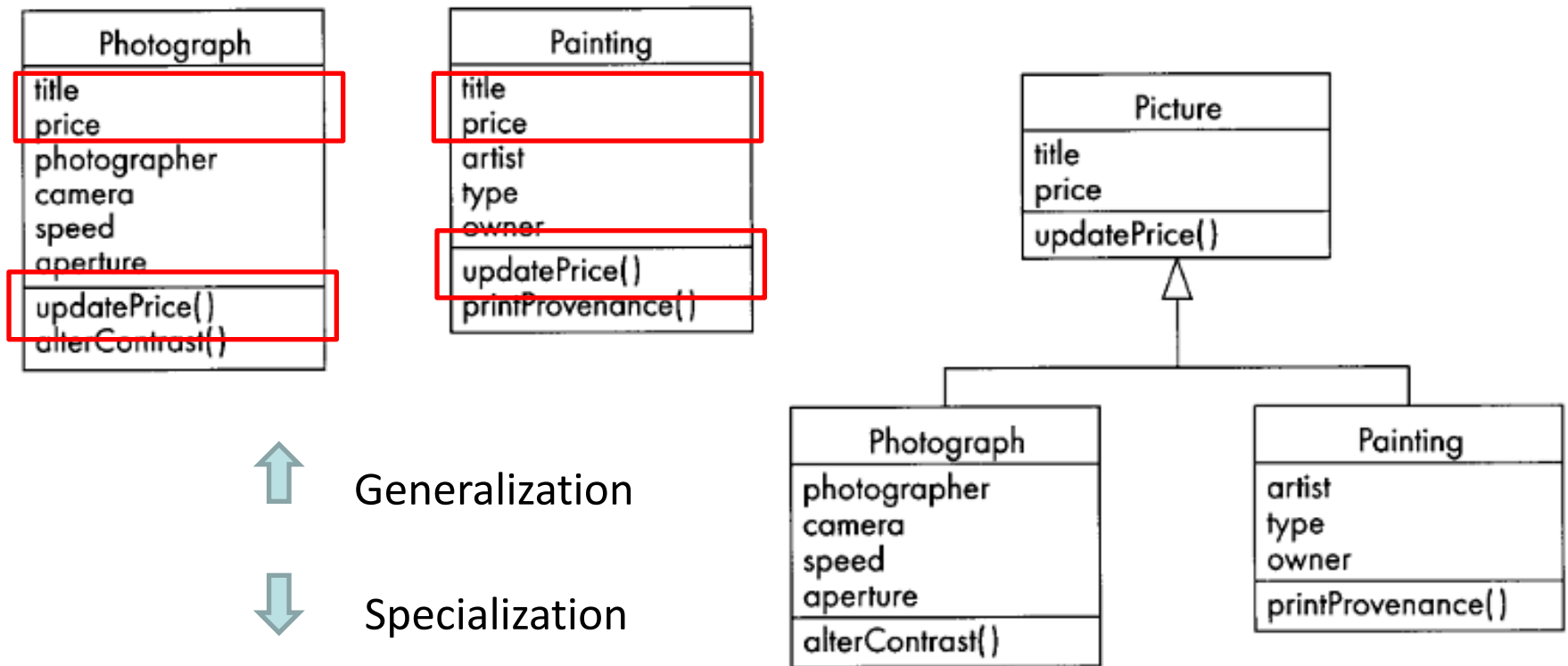


Ví dụ

- Xác định quan hệ giữa các lớp sau nên là gộp (composition) hay kết tập (aggregation)?
 1. “Đơn hàng” và “Chi tiết đơn hàng”
 2. “Phòng ban” và “Nhân viên”
-

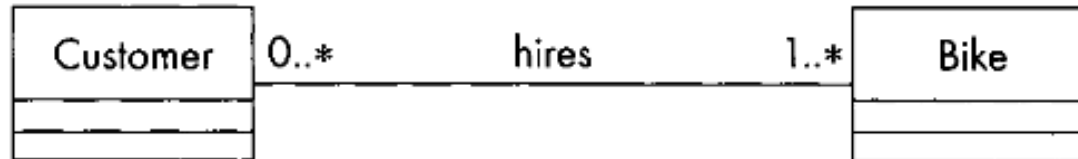
Kế thừa

- Là quan hệ “là một” (is a) hoặc “là một loại” (is a kind of)



Lớp kết hợp

- Ví dụ:



- Có nhận xét gì về liên kết “hires”?
 - Liên kết “hires” bao gồm nhiều thuộc tính (startDate, endDate)
 - Đặt các thuộc tính này ở Customer hay Bike đều không hợp lý
- Trong trường hợp này nên chuyển nó thành một *lớp kết hợp*

Thế nào là một lớp tốt?

- 1. Problem domain:** Các lớp nên phản ánh đúng các đối tượng trong phạm vi bài toán

Ví dụ: Nếu đang phân tích hệ thống thư viện, lớp Book hay Member là tốt, nhưng lớp System hay LibraryManagement lại quá chung chung

- 2. Functionality:** Một lớp cần có cả dữ liệu và hành vi, tránh các lớp chỉ toàn dữ liệu hoặc chỉ toàn hành vi

Ví dụ: Phải có dữ liệu gắn với hành vi xử lý dữ liệu đó → tránh biến lớp thành “struct dữ liệu” hoặc “tập hợp các hàm” rời rạc

- 3. Cohesion:** Lớp phải có tính cố kết cao/ kết dính cao (high cohesion) → mỗi lớp nên tập trung vào chỉ một trách nhiệm chính
-

Ví dụ

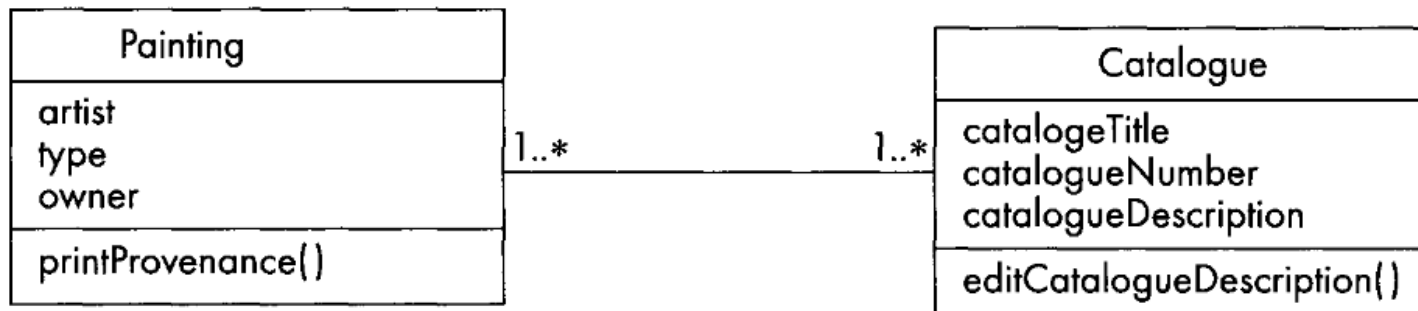
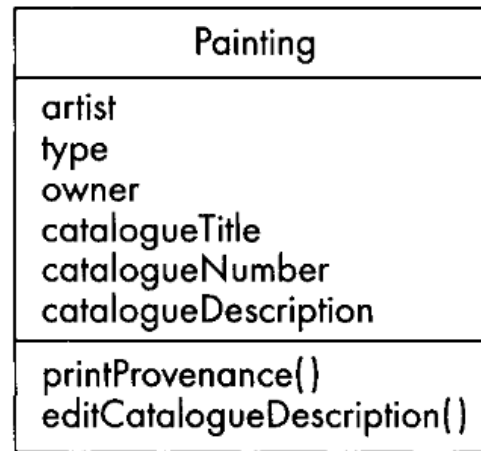
- Lớp sau có tốt không?

Painting
artist type owner
catalogueTitle catalogueNumber catalogueDescription
printProvenance() editCatalogueDescription()

- Không có tính kết dính cao!

Lớp Painting không nên chứa thông tin về Catalogue (danh mục) vì một bức tranh có thể xuất hiện trong nhiều danh mục khác nhau, hoặc không có trong danh mục nào

Tách lớp để đạt kết dính cao



Mô hình bài toán và mô hình phân tích

- Mô hình hóa miền bài toán (**Domain Modeling**)
 - Thực tế đời thực
 - Chỉ gồm các thực thể (Entity class)
 - Mô hình hóa phân tích (**Analysis Modeling**)
 - Thực hiện sau khi có mô hình miền bài toán
 - Cấu trúc sơ bộ của phần mềm
 - Thêm các lớp biên (Boundary class) và lớp điều khiển (Control class) để thực hiện ca sử dụng
-

Ba loại lớp BCE

1. **Boundary** (Lớp biên):

- Nằm ở biên hệ thống, giao tiếp giữa Tác nhân và hệ thống
- Ví dụ: Giao diện người dùng (Form, Web page), cổng kết nối

2. **Control** (Lớp điều khiển):

- "Bộ não" điều phối thực hiện kịch bản của một ca sử dụng
- Ví dụ: Xử lý mượn sách

3. **Entity** (Lớp thực thể):

- Chính là các lớp đã xác định từ trong miền bài toán
 - Ví dụ: Book, Member, Bike
-

Quy tắc tương tác trong BCE

- Để hệ thống dễ bảo trì, các lớp phải tuân thủ một số quy tắc:
 - Tác nhân (Actor) chỉ tương tác với lớp biên (Boundary)
 - Boundary chỉ gửi yêu cầu đến lớp điều khiển (Control) mà không gọi trực tiếp đến Entity
 - Control có quyền tương tác với tất cả: Boundary (trả kết quả), Entity (lấy dữ liệu), và các Control khác
 - Lớp thực thể (Entity) không chủ động gọi Boundary hay Control mà nó chỉ phản hồi khi được yêu cầu
-

Lợi ích của BCE

- High Cohesion

- Kết dính (cố kết) cao: Mỗi lớp chỉ có trách nhiệm về một việc chính
- Lớp Entity chỉ lo về dữ liệu, lớp Control chỉ lo về logic, lớp Boundary chỉ lo về hiển thị

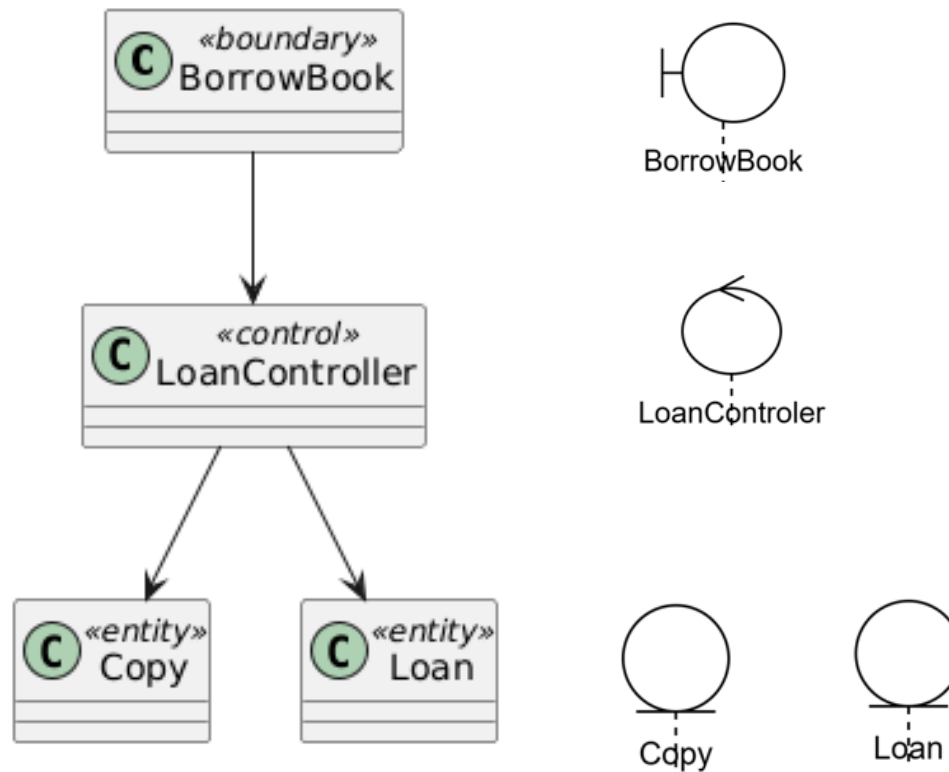
- Low Coupling

- Tương liên yếu: Sự thay đổi lớp này ít ảnh hưởng đến những lớp khác
 - Việc thay đổi giao diện (Boundary) không làm ảnh hưởng đến logic nghiệp vụ (Control) và ngược lại
-

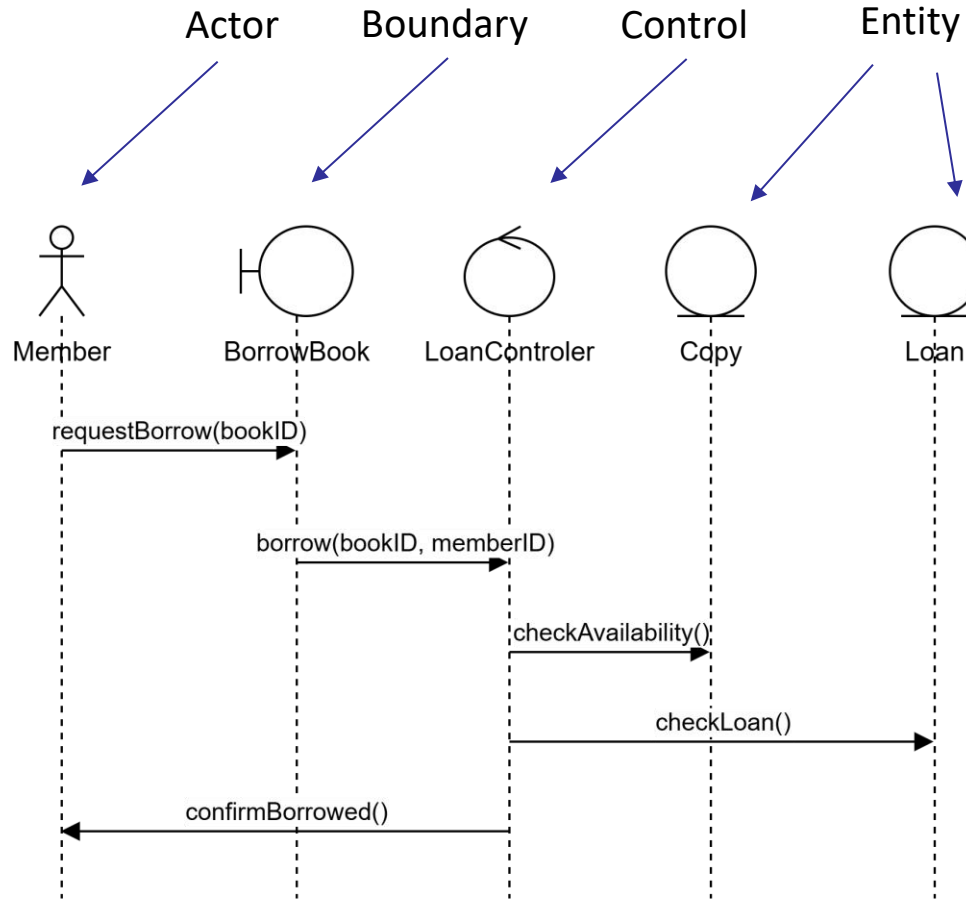
Ví dụ

- Trong “Quản lý thư viện”, chỉ có các lớp Book, Copy, Member, Loan (với các hàm docTep(), ghiTep, muonSach(), ...) thì có được không?
 - Về mặt kỹ thuật là có thể được nhưng chỉ nên với dự án rất nhỏ.
 - Vì hệ quả:
 1. Vi phạm tính Kết dính cao. Lớp Member vừa phải lo quản lý tên, tuổi, vừa phải lo "biết" cách mở tệp tin, đọc dữ liệu, ...
 2. Vi phạm tính Tương liên yếu. Ví dụ: Nếu đổi giao diện từ màn hình Console sang Web → phải tìm và sửa các lệnh in nằm rải rác trong các lớp này
-

Sơ đồ lớp



Ví dụ



Câu hỏi

1. Mục tiêu của phân tích đối tượng là gì?
 2. Sự khác nhau giữa đối tượng và lớp? Ký hiệu UML của chúng?
 3. Các đối tượng giao tiếp với nhau như thế nào?
 4. Hãy trình bày về 4 loại liên kết giữa các lớp?
 5. Trình bày về một số tiêu chuẩn để coi một lớp là tốt?
 6. Tại sao lớp điều khiển (Control) lại giúp cho hệ thống dễ thay đổi giao diện hơn?
 7. Trong mô hình BCE, những loại lớp nào chịu trách nhiệm lưu trữ thông tin lâu dài?
-